# ۱. معماری یکپارچه و میکروسرویس؛ مسئله و ساختار

تحقیق مستند فارسی — تاریخ بررسی منابع: ۱۲ اکتبر ۲۰۲۶

## چکیده و روش تحقیق

یک فروشگاه اینترنتی را می‌توان در قالب یک برنامه ساخت یا مسئولیت‌های آن را میان چند سرویس تقسیم کرد. این انتخاب بر شیوهٔ انتشار، نگهداری داده و حتی پیدا کردن علت یک خطا اثر می‌گذارد. این پژوهش با مراجعه به مستندات رسمی OpenID، OWASP، AWS، Microsoft و IETF و نوشته‌های تخصصی، شرایط مناسب هر گزینه را بررسی می‌کند. ارجاع‌های شماره‌دار به فهرست منابع پایان گزارش اشاره دارند. مثال فروشگاه و مدل هزینه، برای توضیح بحث ساخته شده‌اند و حاصل آزمایش روی یک سامانهٔ واقعی نیستند.

## تعریف و تفاوت ساختاری

در معماری یکپارچه یا Monolithic، قابلیت‌های اصلی برنامه در یک واحد قابل استقرار ارائه می‌شوند. یکپارچه بودن به معنی یک فایل، کد نامرتب یا یک سرور نیست؛ برنامه می‌تواند لایه‌ها و ماژول‌های منظم داشته باشد و چند نسخهٔ همسان از آن اجرا شود. تغییر یک بخش معمولاً به ساخت و انتشار همان واحد برنامه منتهی می‌شود. در معماری Microservices، سامانه از سرویس‌هایی تشکیل می‌شود که پیرامون قابلیت‌های کسب‌وکار سازمان یافته‌اند و از طریق قراردادهای شبکه‌ای ارتباط دارند. استقلال استقرار، معیار مهم‌تری از تعداد خطوط کد یا کوچک بودن نام سرویس است. [۱، ۲]

در میکروسرویس، هر سرویس مسئول داده و رفتار حوزهٔ خود است. سرویس سفارش نباید برای تغییر موجودی مستقیماً جدول سرویس انبار را دست‌کاری کند. مرز مالکیت داده لزوماً به معنی یک ماشین فیزیکی جدا برای هر پایگاه داده نیست؛ مهم، جلوگیری از وابستگی پنهان به ساختار داخلی سرویس دیگر است. انتخاب زبان‌های مختلف ممکن است، اما الزام معماری نیست. [۳]

## یک مثال مشترک برای مقایسه

یک فروشگاه آموزشی را با سه قابلیت «کاتالوگ»، «سفارش» و «انبار» فرض می‌کنیم. در طراحی یکپارچه، این قابلیت‌ها ماژول‌های یک برنامه‌اند و ارتباطشان فراخوانی داخلی است. در طراحی میکروسرویس، سه سرویس مستقل با API یا پیام ارتباط می‌گیرند. پرسش معماری این است: آیا منفعت استقلال این سه بخش، هزینهٔ ارتباط شبکه‌ای و هماهنگی داده را توجیه می‌کند؟ این مثال در ادامه برای توضیح تصمیم‌ها به کار می‌رود.

نقشهٔ مفهومی یکپارچه: کاربر ← برنامهٔ واحد شامل سه ماژول ← ذخیره‌ساز
نقشهٔ مفهومی میکروسرویس: کاربر ← درگاه ← سرویس‌های مستقل و دادهٔ تحت مالکیت هر سرویس

# ۲. امنیت، احراز هویت و کنترل دسترسی

## مفاهیم مشترک در هر دو معماری

احراز هویت مشخص می‌کند درخواست‌کننده کیست؛ مجوزدهی مشخص می‌کند چه کاری روی کدام منبع مجاز است. داشتن نقش «کاربر» به‌تنهایی مجوز دیدن سفارش شخص دیگر نیست. اصل کمترین دسترسی، رد پیش‌فرض و بررسی مجوز در هر درخواست، در هر دو معماری کاربرد دارد. در سامانهٔ چندمستاجری، سازمان مالک داده باید در تصمیم دسترسی لحاظ شود و از شناسهٔ ارسالی کاربر به‌تنهایی نمی‌توان به‌عنوان اثبات مالکیت استفاده کرد. [۴]

## الگوی رایج در برنامهٔ یکپارچه

ورود کاربر می‌تواند یک نشست سمت سرور ایجاد کند و مرورگر فقط شناسهٔ نشست را در کوکی نگه دارد. محافظت کوکی با Secure و HttpOnly، سیاست مناسب SameSite، انقضای نشست و تعویض شناسه پس از ورود یا تغییر سطح دسترسی مهم‌اند. نشست معتبر همچنان به بررسی مجوز نیاز دارد. تمرکز این کنترل‌ها در یک برنامه، هماهنگی را ساده‌تر می‌کند؛ در عوض، خطای مرکزی ممکن است دامنهٔ اثر بزرگی داشته باشد. [۵]

هر دو معماری می‌توانند از ارائه‌دهندهٔ هویت مشترک استفاده کنند. OAuth 2.0 چارچوب تفویض دسترسی است و OpenID Connect لایهٔ احراز هویت را روی آن تعریف می‌کند. ID Token برای بیان هویت نزد کلاینت است؛ نباید بدون طراحی و اعتبارسنجی مناسب جایگزین توکن دسترسی API شود. یکپارچه بودن برنامه استفاده از ورود یکپارچه را منع نمی‌کند. [۶]

## چالش‌های افزوده در میکروسرویس

درگاه می‌تواند درخواست نامعتبر را در ورودی رد کند، اما مجوز عملیات وابسته به سفارش یا سازمان باید در مرز سرویس نیز اعمال شود. هویت سرویس فراخواننده و هویت کاربر دو موضوع جدا هستند. mTLS برای شناسایی طرفین ارتباط و حفاظت انتقال کاربرد دارد؛ مدیریت، چرخش و لغو گواهی‌ها هزینهٔ عملیاتی دارد. اعتماد صرف به «شبکهٔ داخلی» کافی نیست. [۷]

JWT صرفاً یک قالب توکن است. پذیرش آن باید به الگوریتم‌های مجاز، امضای معتبر، صادرکننده و مخاطب مورد انتظار وابسته باشد؛ الزامات زمان انقضا نیز باید مطابق نوع توکن اعمال شود. بررسی امضا به‌تنهایی مجوز کسب‌وکار ایجاد نمی‌کند. اعتبارسنجی محلی، وابستگی شبکه‌ای را کم می‌کند اما لغو فوری توکن را تضمین نمی‌کند؛ اعتبارسنجی برخط نیز به دسترس‌پذیری سرویس هویت وابسته است. [۷، ۸]

در فروشگاه فرضی، بررسی امنیت باید دسترسی به سفارش شخص دیگر، جداسازی دادهٔ سازمان‌ها و احتمال ثبت توکن در گزارش‌ها را پوشش دهد. رعایت این توصیه‌های عمومی برای تأیید امنیت یک محصول کافی نیست؛ صحت اجرای آنها نیز باید آزمایش شود.

# ۳. مقیاس‌پذیری، کارایی و تحمل خطا

## مقیاس‌پذیری با سرعت یک درخواست فرق دارد

مقیاس‌پذیری یعنی پاسخ‌گویی به رشد بار با حفظ هدف‌های خدمت؛ کارایی به زمان و منابع مصرفی برای انجام کار مربوط است. برنامهٔ یکپارچه می‌تواند با منابع قوی‌تر یا افزایش تعداد نسخه‌ها رشد کند. در میکروسرویس می‌توان فقط بخش پرتقاضا را تکثیر کرد، اما سربار شبکه، تبدیل داده و هماهنگی ممکن است زمان پاسخ را افزایش دهد. انتخاب معماری به‌خودی‌خود تضمین سرعت نیست. [۱]

در فروشگاه فرضی، اگر بیشتر بار مربوط به جست‌وجوی کاتالوگ باشد، افزایش نسخه‌های همان سرویس می‌تواند مفید باشد. اما اگر یک پرس‌وجوی ناکارآمد یا قفل پایگاه داده عامل کندی باشد، افزودن سرویس حتی ممکن است فشار را بیشتر کند. بنابراین پیش از تجزیهٔ برنامه باید اندازه گرفت که وقت و منابع دقیقاً کجا مصرف می‌شوند.

## خودکارسازی رشد ظرفیت

برای نمونه، Horizontal Pod Autoscaler در Kubernetes تعداد نسخه‌ها را با توجه به معیارهای تنظیم‌شده مانند مصرف منابع یا معیار سفارشی تغییر می‌دهد. این سازوکار به دادهٔ پایش و پیکربندی درست وابسته است و ظرفیت نامحدود ایجاد نمی‌کند. زمان راه‌اندازی نسخهٔ جدید و رفتار کاهش ظرفیت نیز باید بررسی شود. Kubernetes یک ابزار ممکن است، نه شرط تعریف میکروسرویس. [۹، ۳]

## خرابی جزئی و انتشار خطا

در سامانهٔ توزیع‌شده ممکن است سرویس مقصد سالم باشد اما پاسخ به‌موقع نرسد. الگوی Circuit Breaker پس از شکست‌های مشخص، موقتاً فراخوانی‌های بعدی را محدود می‌کند و بعد امکان بازیابی را می‌سنجد. این الگو با تلاش مجدد تفاوت دارد و خودِ مشکل سرویس مقصد را تعمیر نمی‌کند. انتخاب آستانه‌ها باید با رفتار واقعی سامانه سنجیده شود. [۱۰]

قطع سرویس پیشنهاد محصول را می‌توان با حذف موقت پیشنهادها مدیریت کرد. نرسیدن پاسخ رزرو موجودی مسئلهٔ دیگری است: شاید رزرو انجام شده باشد و فقط پاسخ نرسیده باشد. برای هر وابستگی باید از قبل مشخص شود برنامه چه مدت منتظر می‌ماند، چند بار تلاش می‌کند و در پایان چه وضعیتی به کاربر نشان می‌دهد.

## چه چیزی باید اندازه‌گیری شود؟

آزمایش منصفانه باید بار یکسان، اندازهٔ دادهٔ مشابه و منابع مشخص داشته باشد. نرخ درخواست موفق، زمان پاسخ صدک ۹۵ و ۹۹، مصرف حافظه، عمق صف و هزینهٔ هر عملیات موفق معیارهای مفیدی برای مثال آموزشی هستند. میانگین زمان پاسخ به‌تنهایی تأخیرهای شدید برخی کاربران را پنهان می‌کند. بدون چنین سنجشی، ادعای «میکروسرویس سریع‌تر است» برای پروژهٔ مشخص قابل دفاع نیست.

# ۴. داده، تراکنش‌ها و وابستگی‌ها

## مرز تراکنش و سازگاری

در طراحی یکپارچه‌ای که داده‌های مرتبط در یک پایگاه داده قرار دارند، چند تغییر می‌توانند در تراکنش محلی انجام شوند. در طراحی میکروسرویس، تغییر هم‌زمان چند ذخیره‌ساز مستقل دشوارتر است. الگوی Saga کار را به تراکنش‌های محلی و عملیات جبرانی تقسیم می‌کند. جبران الزاماً بازگرداندن فیزیکی تاریخچه نیست؛ می‌تواند یک عملیات کسب‌وکاری مانند آزادکردن رزرو باشد. [۱۱]

فرض کنیم سفارش ابتدا «در انتظار رزرو» باشد و پس از رزرو موجودی تأیید شود. اگر رزرو شکست بخورد، سفارش باید وضعیت روشنی پیدا کند؛ رها شدن آن در یک حالت نامعلوم برای کاربر و پشتیبانی مشکل‌ساز است. اگر قواعد کسب‌وکار هیچ فاصله‌ای میان این دو تغییر را نپذیرند، هزینهٔ جداسازی سفارش و انبار را باید با دقت بیشتری سنجید.

## پیام‌رسانی قابل اتکا

ثبت موفق داده و شکست انتشار پیام، مسئلهٔ دو نوشتن جداگانه است. Transactional Outbox تغییر داده و رکورد پیام خروجی را در یک تراکنش محلی ثبت می‌کند و ارسال پیام بعداً انجام می‌شود. امکان پیام تکراری همچنان وجود دارد، بنابراین مصرف‌کننده باید تکرار را مدیریت کند. این الگو به‌تنهایی تضمین «فقط یک‌بار انجام شدن همهٔ آثار خارجی» نمی‌دهد. [۱۲]

اگر رویداد رزرو دوباره برسد، نباید موجودی دوباره کم شود. برای این مثال می‌توان شناسهٔ عملیات را نگه داشت و پیش از اعمال تغییر، تکراری بودن آن را بررسی کرد. پیام‌های ناموفق نیز به قاعدهٔ نگهداری و مسئول رسیدگی نیاز دارند. وجود صف به‌تنهایی روشن نمی‌کند با سفارش حل‌نشده چه باید کرد.

## وابستگی کد، قرارداد و تیم

میکروسرویس استقلال را از راه قرارداد به دست می‌آورد. تغییر ناسازگار API، اشتراک مستقیم جدول‌ها یا اجبار به انتشار هم‌زمان سرویس‌ها این استقلال را کاهش می‌دهد. چنین سامانه‌ای ممکن است هزینهٔ توزیع‌شدگی را داشته باشد ولی همچنان مثل یک واحد بزرگ تغییر کند. مرزهای مناسب باید با قابلیت کسب‌وکار و مسئولیت تیم هماهنگ باشند. [۲]

وجود همهٔ سرویس‌ها در یک مخزن Git لزوماً استقلال انتشار را از بین نمی‌برد؛ تعداد مخزن‌ها با تعداد واحدهای استقرار یک مفهوم نیست. برای مثال آموزشی، مهم‌تر از انتخاب یک یا چند مخزن این است که بتوان سرویس انبار را با قرارداد سازگار آزمایش و منتشر کرد، بدون آنکه سرویس سفارش مجبور به انتشار تازه شود. کتابخانهٔ مشترک برای ابزار عمومی می‌تواند مفید باشد، ولی اشتراک مدل داخلی دامنه نیازمند بررسی دقیق وابستگی است.

# ۵. استقرار، تست و خطایابی

## انتشار و بازگشت نسخه

در میکروسرویس، خط ساخت و انتشار باید استقلال سرویس‌ها و سازگاری نسخه‌ها را حفظ کند. استقرار تدریجی و پایش سلامت می‌تواند ریسک را کاهش دهد، اما تعداد بیشتر خطوط انتشار، پیکربندی‌ها و محیط‌ها کار عملیاتی بیشتری ایجاد می‌کند. بازگشت نسخهٔ برنامه همیشه تغییر پایگاه داده را برنمی‌گرداند. باید نسخه‌های قدیم و جدید در بازهٔ گذار با قرارداد و دادهٔ موجود سازگار باشند. [۱۳]

در نمونهٔ آموزشی، برای افزودن یک فیلد سفارش، ابتدا پذیرش دادهٔ قدیم حفظ می‌شود، سپس تولیدکننده تغییر می‌کند و حذف شکل قدیمی به مرحلهٔ بعد موکول می‌شود. این ترتیب پیشنهادی برای کاهش شکست مصرف‌کنندگان است. در برنامهٔ یکپارچه هم مهاجرت پایگاه داده و نسخه‌های هم‌زمان مسئله‌اند؛ تفاوت، تعداد مرزهایی است که باید هماهنگ شوند.

## لایه‌های آزمون

آزمون واحد منطق کوچک را می‌سنجد؛ آزمون یکپارچگی ارتباط با پایگاه داده یا سرویس وابسته را بررسی می‌کند؛ آزمون قرارداد، سازگاری انتظارات مصرف‌کننده و ارائه‌دهنده را هدف می‌گیرد. آزمون سرتاسری مسیر کامل کاربر را می‌سنجد، اما در سامانهٔ چندسرویسی معمولاً کندتر و وابسته‌تر به محیط است. جایگزین‌های آزمایشی سریع‌اند ولی باید با رفتار واقعی وابستگی هم‌خوان بمانند. [۱۴]

طرح آزمون پیشنهادی فروشگاه شامل محاسبهٔ سفارش در سطح واحد، رزرو موجودی در سطح یکپارچگی، قرارداد پاسخ انبار و یک مسیر سرتاسری خرید است. افزون بر مسیر موفق، تکرار پیام، تأخیر پاسخ و رد دسترسی بررسی می‌شود. این فهرست طرح پژوهشی است و ادعای اجرای این آزمون‌ها روی نرم‌افزار واقعی ندارد.

## مشاهده‌پذیری و تشخیص ریشهٔ خطا

گزارش رویداد، معیار عددی و ردگیری درخواست سه نوع دادهٔ مکمل‌اند. ردگیری توزیع‌شده مسیر درخواست میان سرویس‌ها را آشکار می‌کند و انتشار زمینهٔ ردگیری، ارتباط بین عملیات را حفظ می‌کند. OpenTelemetry ابزارها و قراردادهایی برای تولید و انتقال این داده‌ها فراهم می‌کند؛ صرف نصب ابزار به معنی یافتن خودکار ریشهٔ مشکل نیست. [۱۵]

برای مثال، «ثبت سفارش کند است» می‌تواند ناشی از صف انبار یا انتظار پایگاه داده باشد. در برنامهٔ یکپارچه، دنبال‌کردن فراخوانی در یک فرایند معمولاً ساده‌تر است. در میکروسرویس، شناسهٔ همبستگی، نسخهٔ سرویس و زمان هر مرحله به مقایسهٔ رویدادها کمک می‌کند. اطلاعات محرمانه باید پیش از ثبت از گزارش حذف شود؛ سامانهٔ متمرکز گزارش نیز به سیاست دسترسی و نگهداری نیاز دارد. [۷، ۱۵]

# ۶. هزینهٔ کل و موازنه‌های انتخاب

## هزینه فقط صورتحساب سرور نیست

هزینهٔ کل مالکیت شامل توسعهٔ اولیه، زیرساخت، پایش، نگهداری، رخدادها و تغییرات آینده است. مقایسهٔ AWS بر سادگی نسبی شروع با برنامهٔ یکپارچه و انعطاف بیشتر توسعه و رشد میکروسرویس تأکید دارد؛ این گزاره به معنی ارزان‌تر بودن قطعی یک گزینه برای همهٔ پروژه‌ها نیست. [۱]

برای برآورد می‌توان هزینهٔ یک دوره را به چهار بخش تقسیم کرد: زیرساخت، کار توسعه و عملیات، رخداد و بازیابی، و مهاجرت. نرخ نیروی انسانی، حجم بار، سطح خدمت و تعداد محیط‌ها باید معلوم باشد. صورت تمرین این اطلاعات را مشخص نکرده است؛ بنابراین نمی‌توان مبلغ یا درصد معتبری برای صرفه‌جویی اعلام کرد.

## جدول مقایسهٔ تصمیم‌محور

| معیار | یکپارچهٔ ماژولار | میکروسرویس |
|---|---|---|
| **واحد انتشار** | برنامهٔ اصلی | هر سرویس، با قرارداد سازگار |
| **رشد ظرفیت** | افزایش منابع یا نسخه‌های برنامه | رشد انتخابی سرویس‌ها |
| **تراکنش مرتبط** | معمولاً ساده‌تر در یک ذخیره‌ساز | هماهنگی میان مالک‌های داده |
| **شروع و محیط توسعه** | اجزای کمتر برای راه‌اندازی | نیاز به مدیریت وابستگی‌ها |
| **خطایابی** | مسیر داخلی قابل دنبال‌کردن | نیاز بیشتر به ردگیری توزیع‌شده |
| **خودمختاری تیم** | نیاز به هماهنگی انتشار مشترک | ممکن، با مرزبندی و مالکیت روشن |
| **ریسک اصلی** | درهم‌تنیدگی ماژول‌ها | پیچیدگی شبکه و وابستگی قراردادی |

جدول، جمع‌بندی تحلیلی بر پایهٔ منابع [۱، ۲، ۱۱، ۱۳، ۱۵] است؛ عبارت‌های آن وابسته به کیفیت پیاده‌سازی‌اند.

## دو سناریوی آموزشی

برای تیم سه‌نفره‌ای با دامنهٔ در حال تغییر و بار متوسط، برنامهٔ یکپارچهٔ ماژولار معمولاً نقطهٔ شروع کم‌هزینه‌تری برای آزمودن محصول است. برای چند تیم مستقل با مرزهای پایدار و بار بسیار متفاوت میان کاتالوگ و سفارش، جداسازی انتخابی ممکن است زمان انتظار برای انتشار را کم کند. تعداد اعضای تیم در این مثال فرض آموزشی است و آستانهٔ علمی انتخاب معماری محسوب نمی‌شود.

# ۷. توصیه‌ها، مهاجرت و جمع‌بندی

## چرا شروع ساده می‌تواند منطقی باشد؟

در نوشتهٔ Monolith First، فاولر پیشنهاد می‌کند هنگام ناشناخته بودن مرزهای دامنه، شروع یکپارچه و جداسازی بعدی بررسی شود. او این توصیه را بر تجربه‌ها و مشاهده‌ها بنا می‌کند، نه یک قانون اثبات‌شده برای همهٔ محصولات. ارزش این دیدگاه، کاهش هزینهٔ تصمیم زودهنگام دربارهٔ مرزهایی است که هنوز مرتب تغییر می‌کنند. [۱۶]

## مسیر پیشنهادی تصمیم‌گیری

۱) نیاز غیرعملکردی را قابل سنجش بنویسید: بار مورد انتظار، زمان پاسخ، دسترس‌پذیری و زمان قابل تحمل بازیابی. ۲) مرز ماژول‌ها و مالک داده را مشخص کنید. ۳) گلوگاه را با شواهد پیدا کنید. ۴) منفعت جداسازی یک قابلیت را در برابر هزینهٔ عملیات و هماهنگی بسنجید. ۵) تصمیم و فرض‌ها را ثبت و پس از تغییر بار یا تیم بازبینی کنید. این مراحل پیشنهاد تحلیلی گزارش‌اند و جایگزین شناخت دامنه نیستند.

## مهاجرت تدریجی

الگوی Strangler Fig انتقال تدریجی قابلیت‌ها به سامانهٔ تازه را پیشنهاد می‌کند. یک لایهٔ هدایت می‌تواند درخواست مربوط به قابلیت منتقل‌شده را به بخش جدید برساند و بقیهٔ درخواست‌ها را به سامانهٔ قبلی بدهد. این روش امکان تغییر مرحله‌ای را فراهم می‌کند، اما دورهٔ همزیستی و وابستگی به لایهٔ هدایت باید مدیریت شود. [۱۷]

در مثال فروشگاه، می‌توان ابتدا قابلیت مستقلی مانند تولید گزارش را جدا کرد؛ البته تنها اگر مالکیت داده، روش دریافت داده و هدف مستقل آن روشن باشد. معیار پذیرش پیشنهادی شامل صحت خروجی، سازگاری قرارداد، هزینهٔ عملیاتی قابل تحمل و امکان بازگشت ترافیک است. تقسیم صرف بر اساس نام جدول‌ها، دلیل کافی برای ایجاد سرویس نیست.

## جمع‌بندی و محدودیت پژوهش

یکپارچهٔ ماژولار برای شروع محصول، هماهنگی ساده‌تر و تغییرات نزدیک به هم گزینه‌ای قابل دفاع است. میکروسرویس زمانی ارزش بیشتری دارد که استقلال تیم و انتشار، رشد نامتوازن قابلیت‌ها یا مرزهای روشن دامنه منفعت ملموس ایجاد کنند. راهکار ترکیبی نیز ممکن است: هستهٔ منظم یکپارچه در کنار چند سرویس مستقل برای نیازهای مشخص. نتیجهٔ این تحقیق، انتخاب ساده‌ترین ساختاری است که نیازهای سنجیده‌شده را برآورده کند و امکان اصلاح آینده را حفظ کند.

این متن مرور کیفی منابع و تحلیل یک مثال فرضی است؛ بنچمارک، مطالعهٔ هزینهٔ واقعی یا ممیزی امنیتی نیست. مستندات فروشندگان برای شناخت الگوها مفیدند، اما توصیه‌های محصولی آنها باید با زمینهٔ پروژه سنجیده شود. امنیت، چندمستاجری و عملیات مالی نیازمند بررسی تخصصی جداگانه‌اند. هیچ‌یک از دو معماری به‌تنهایی تضمین امنیت، انطباق یا کاهش هزینه نمی‌دهد.

## ۸. منابع و مآخذ

تاریخ دسترسی به همهٔ منابع: ۱۲ اکتبر ۲۰۲۶. عنوان و نشانی اصلی برای بازیابی منبع آمده است. متن گزارش بازنویسی و تحلیل فارسی است؛ نقل‌قول مستقیم طولانی استفاده نشده است.

[1] AWS — Monolithic vs. Microservices
https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/

[2] James Lewis & Martin Fowler — Microservices (2014)
https://martinfowler.com/articles/microservices.html

[3] Microsoft Azure Architecture Center — Microservices architecture style
https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices

[4] OWASP — Authorization Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[5] OWASP — Session Management Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[6] OpenID Foundation — OpenID Connect Core 1.0, errata set 2 (2023)
https://openid.net/specs/openid-connect-core-1_0.html

[7] OWASP — Microservices Security Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/Microservices_Security_Cheat_Sheet.html

[8] IETF / RFC Editor — RFC 8725: JSON Web Token Best Current Practices (2020)
https://www.rfc-editor.org/rfc/rfc8725.html

[9] Kubernetes — Horizontal Pod Autoscaling
https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

# منابع و مآخذ — ادامه

تاریخ دسترسی به همهٔ منابع: ۱۲ اکتبر ۲۰۲۶. عنوان و نشانی اصلی برای بازیابی منبع آمده است. متن گزارش بازنویسی و تحلیل فارسی است؛ نقل‌قول مستقیم طولانی استفاده نشده است.

[10] Microsoft Azure Architecture Center — Circuit Breaker pattern
https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker

[11] Microsoft Azure Architecture Center — Saga distributed transactions pattern
https://learn.microsoft.com/en-us/azure/architecture/patterns/saga

[12] AWS Prescriptive Guidance — Transactional outbox pattern
https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/transactional-outbox.html

[13] Microsoft Azure Architecture Center — CI/CD for microservices
https://learn.microsoft.com/en-us/azure/architecture/microservices/ci-cd

[14] Toby Clemson / MartinFowler.com — Testing Strategies in a Microservice Architecture (2014)
https://martinfowler.com/articles/microservice-testing/

[15] OpenTelemetry — Observability primer
https://opentelemetry.io/docs/concepts/observability-primer/

[16] Martin Fowler — Monolith First (2015)
https://martinfowler.com/bliki/MonolithFirst.html

[17] Microsoft Azure Architecture Center — Strangler Fig pattern
https://learn.microsoft.com/en-us/azure/architecture/patterns/strangler-fig